Name: Gunjan Rupani
Batch: T22
Roll No.:2401096
Subject: ASD&D

Experiment 9: Provisioning AWS Infrastructure Using Terraform

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

1. Aim

To provision, configure, and manage an AWS EC2 instance using Terraform by implementing the Infrastructure as Code (IaC) approach.

2. Theory

Terraform is an open-source Infrastructure as Code tool developed by HashiCorp. It allows users to define, provision, and manage cloud infrastructure using configuration files written in HashiCorp Configuration Language (HCL).

Amazon Web Services (AWS) is a cloud computing platform that provides services such as virtual machines, storage, networking, and databases.

Amazon EC2 (Elastic Compute Cloud) provides scalable virtual servers in the cloud. Terraform can be used to create and manage EC2 instances automatically without manually configuring every resource through the AWS Management Console.

Step 1: Create or Verify an AWS Account

1. Open the AWS Management Console.
2. Sign in to the AWS account.
3. Open the EC2 Dashboard.
4. Verify that the account has the required permissions to create EC2 instances and use networking resources.

Step 2: Install Terraform on Windows

1. Visit the official Terraform installation page.
2. Download the Windows AMD64 ZIP package.
3. Extract terraform.exe into a folder such as C:\Terraform.
4. Add C:\Terraform to the Windows PATH environment variable.
5. Open a new PowerShell window and verify the installation using:

```
terraform -version
```

Step 3: Install AWS CLI

1. Download the AWS CLI Windows MSI installer from the official AWS documentation.

2. Install AWS CLI on the Windows system.
3. Open PowerShell and verify the installation using:

```
aws --version
```

Step 4: Configure AWS Credentials

1. Create an IAM user with the required permissions for the experiment.
2. Configure AWS credentials using an approved authentication method.
3. Execute the following command:

```
aws configure
```

4. Enter the required AWS credentials, default region (us-east-1), and output format (json).
5. Verify the AWS connection using:

```
aws sts get-caller-identity
```

A successful response confirms that AWS CLI can authenticate with the configured identity.

Step 5: Create the Terraform Configuration

1. Create a folder named C:\TerraformLab.
2. Open PowerShell and navigate to the folder:

```
mkdir C:\TerraformLab
cd C:\TerraformLab
```

3. Create a file named main.tf.
4. Define the AWS provider and EC2 instance configuration in the file.

Step 6: Initialize Terraform

Execute the following command:

```
terraform init
```

This command initializes the Terraform working directory and downloads the required AWS provider plugin.

Step 7: Validate the Configuration

Execute:

```
terraform fmt
terraform validate
```

The terraform fmt command formats the configuration file, while terraform validate checks whether the configuration syntax and structure are valid.

Step 8: Generate the Execution Plan

Execute:

terraform plan

Terraform examines the configuration and displays the resources that will be created, modified, or destroyed.

Step 9: Create the EC2 Instance

Execute:

terraform apply

Review the proposed changes and type yes when prompted.

Terraform communicates with AWS and provisions the EC2 instance according to the configuration file.

Step 10: Display Resource Outputs

Create an outputs.tf file in the same folder to display the EC2 instance ID, public IP address, and instance type.

Execute:

terraform apply
terraform output

The output values help verify the details of the provisioned instance.

Step 11: Verify and Destroy the Resources

1. Open the AWS Management Console.
2. Navigate to EC2 → Instances.
3. Verify the instance name, instance ID, instance type, and instance state.
4. Capture clear screenshots of the Terraform commands and AWS console.
5. Execute the following command to remove the resources created by the experiment:

terraform destroy

6. Review the proposed deletion and type yes when prompted.
7. Verify that the instance has been terminated.

Note: All .tf files in the same folder are treated as one Terraform configuration. The filename does not determine which configuration gets executed.

3. Terraform Configuration Code

Create a file named main.tf in C:\TerraformLab and enter the following code:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "student_vm" {
  ami           = "ami-0c02fb55956c7d316"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Student-VM"
  }
}
```

Explanation of the Code

1. **Terraform block:** Specifies the required AWS provider and its version.
2. **Provider block:** Configures Terraform to use AWS in the us-east-1 region.
3. **Resource block:** Defines an EC2 instance named student_vm.
4. **AMI:** Specifies the Amazon Machine Image used to launch the virtual machine.
5. **Instance type:** Specifies t2.micro as the virtual machine type.
6. **Tags:** Assigns the name Terraform-Student-VM to the instance.

The AMI must be valid and available for the selected AWS region and account.

4. Output Configuration Code

Create a file named outputs.tf in the same folder and enter:

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.student_vm.id
}

output "public_ip" {
  description = "Public IP address, if assigned"
  value       = aws_instance.student_vm.public_ip
}

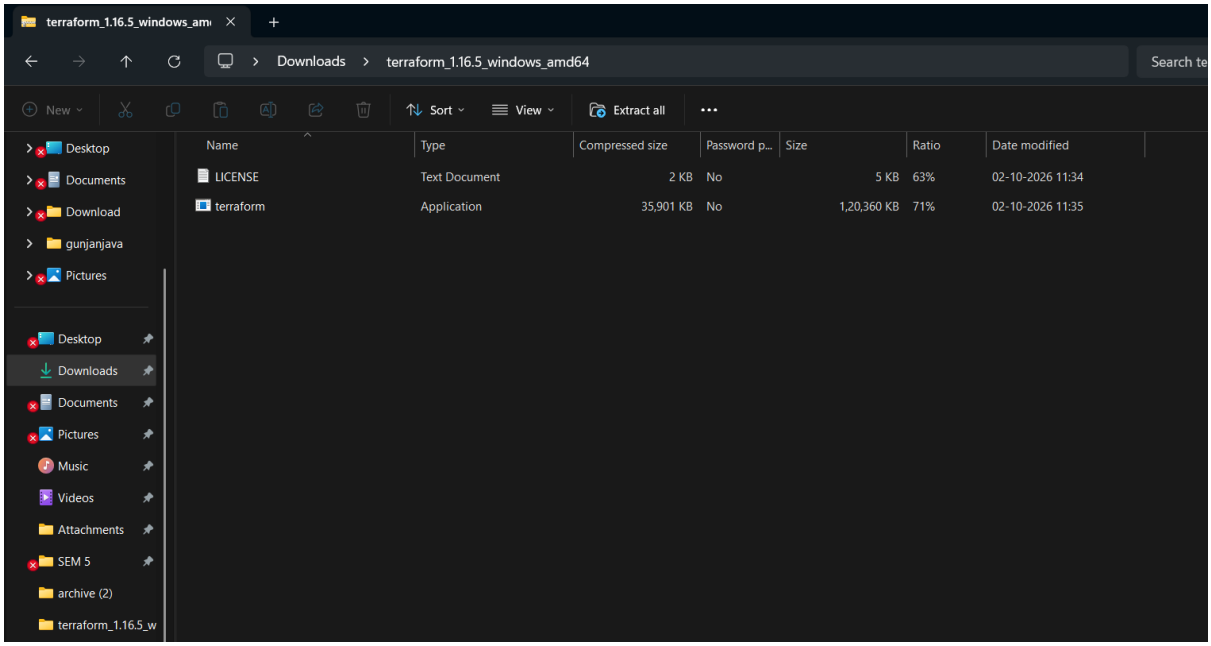
output "instance_type" {
```

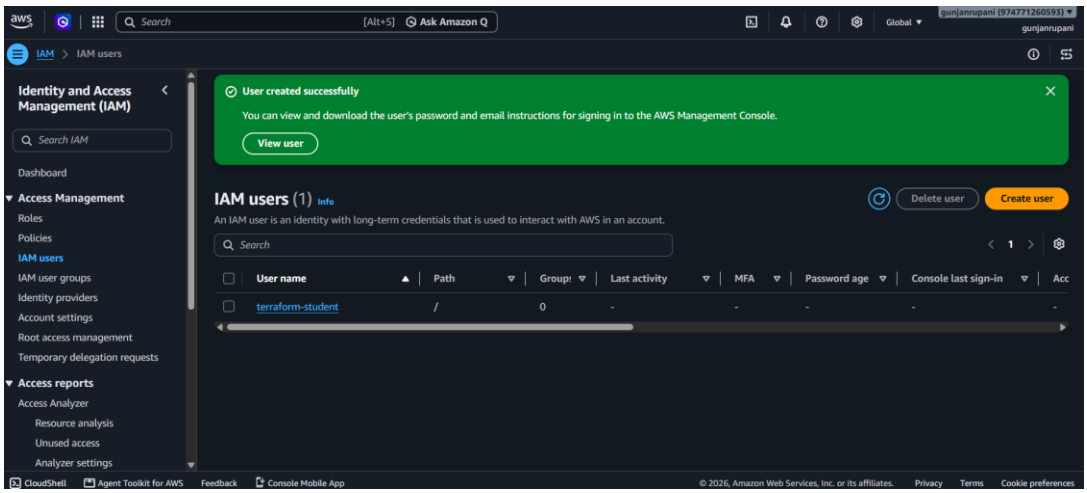
```
  value = aws_instance.student_vm.instance_type
}
```

Explanation

- instance_id displays the unique identifier assigned to the EC2 instance.
- public_ip displays the public IPv4 address if one is assigned.
- instance_type displays the selected EC2 instance type.

6. Output





```
Default output format [None]: json
PS C:\Users\gunja> aws sts get-caller-identity
{
    "UserId": "AIDA6F5HFBSY6233I355O",
    "Account": "974771260593",
    "Arn": "arn:aws:iam::974771260593:user/terraform-student"
}

PS C:\Users\gunja>
```

```
Windows PowerShell

PS C:\Users\gunja> cd C:\TerraformLab
PS C:\TerraformLab> notepad main.tf
PS C:\TerraformLab> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.68.0...
- Installed hashicorp/aws v6.68.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

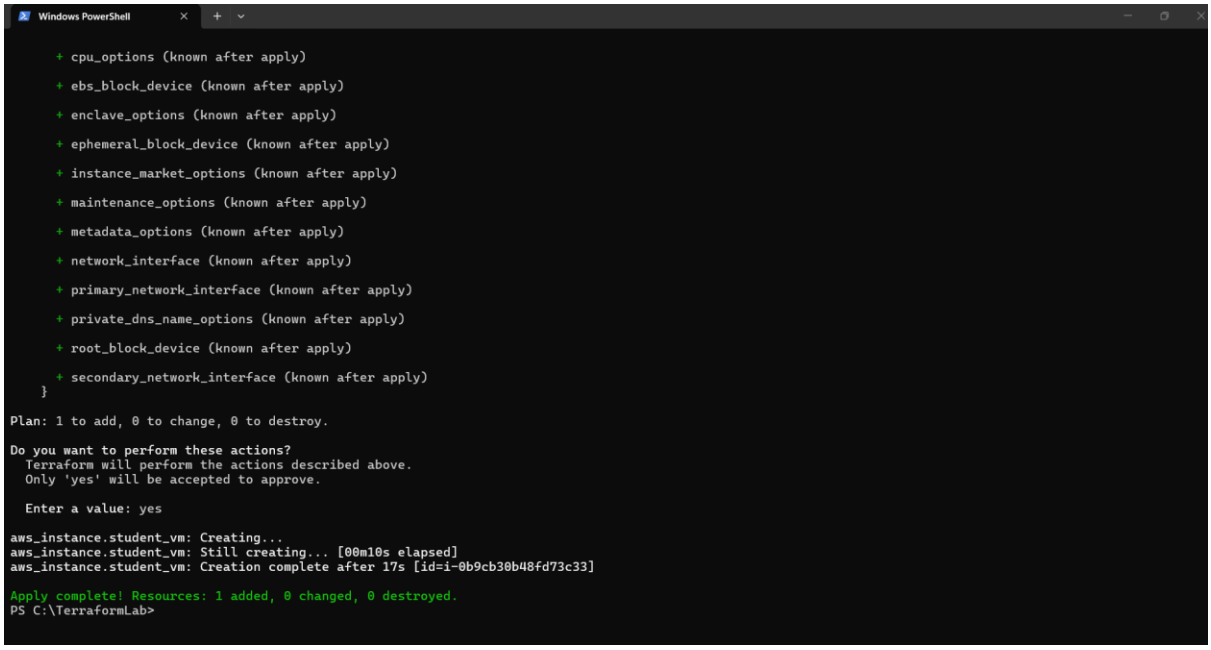
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\TerraformLab> terraform validate
Success! The configuration is valid.

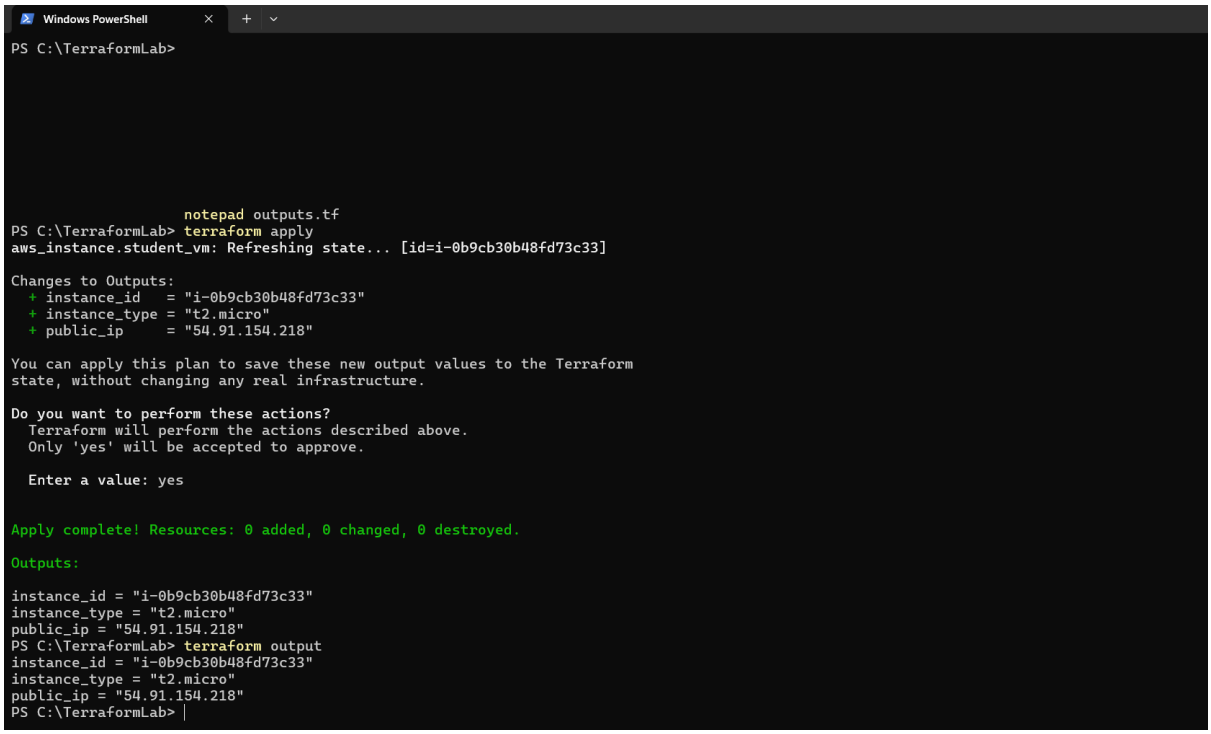
PS C:\TerraformLab>
PS C:\TerraformLab> terraform plan

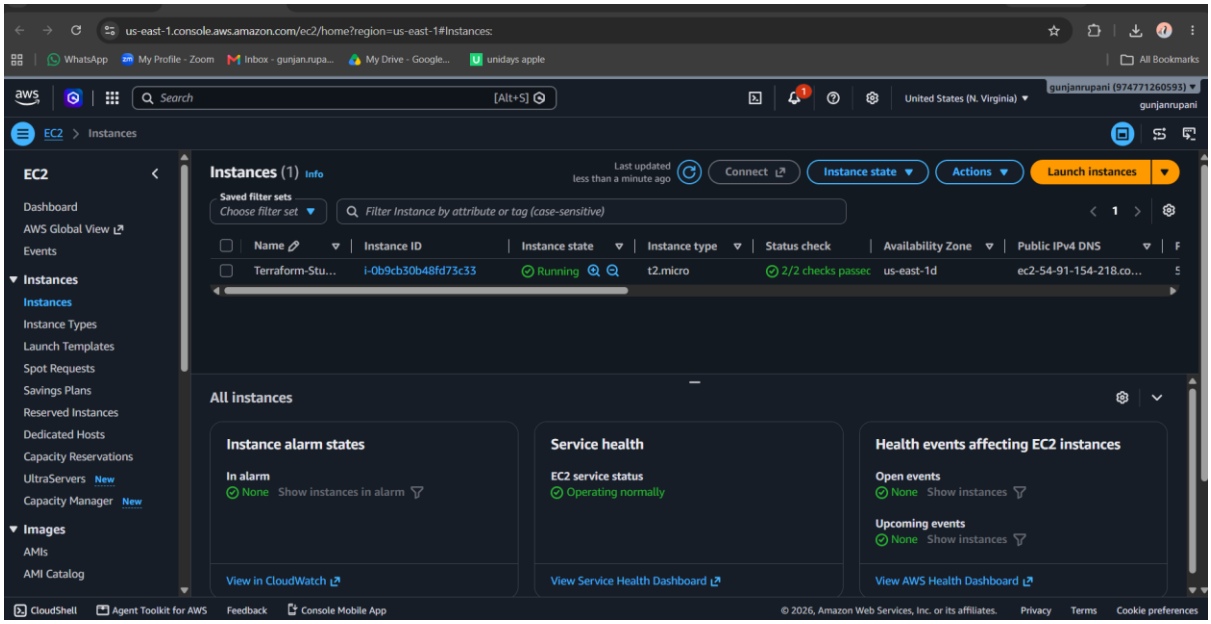
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

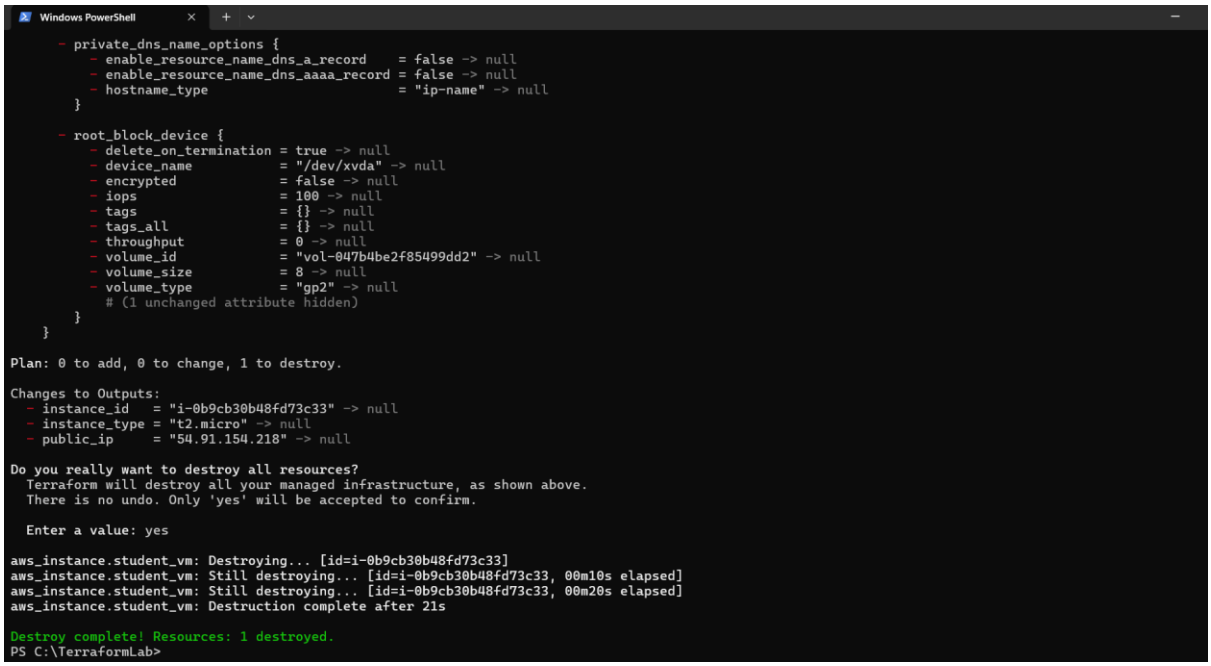
Terraform will perform the following actions:

  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
```


Windows PowerShell
+ cpu_options (known after apply)
+ ebs_block_device (known after apply)
+ enclave_options (known after apply)
+ ephemeral_block_device (known after apply)
+ instance_market_options (known after apply)
+ maintenance_options (known after apply)
+ metadata_options (known after apply)
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.student_vm: Creating...
aws_instance.student_vm: Still creating... [00m10s elapsed]
aws_instance.student_vm: Creation complete after 17s [id=i-0b9cb30b48fd73c33]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\TerraformLab>


Windows PowerShell
PS C:\TerraformLab>
notepad outputs.tf
PS C:\TerraformLab> terraform apply
aws_instance.student_vm: Refreshing state... [id=i-0b9cb30b48fd73c33]
Changes to Outputs:
+ instance_id = "i-0b9cb30b48fd73c33"
+ instance_type = "t2.micro"
+ public_ip = "54.91.154.218"
You can apply this plan to save these new output values to the Terraform
state, without changing any real infrastructure.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
Outputs:
instance_id = "i-0b9cb30b48fd73c33"
instance_type = "t2.micro"
public_ip = "54.91.154.218"
PS C:\TerraformLab> terraform output
instance_id = "i-0b9cb30b48fd73c33"
instance_type = "t2.micro"
public_ip = "54.91.154.218"
PS C:\TerraformLab>





7. Learning Outcomes Achieved

Students gained practical knowledge of using Terraform to define, provision, and manage AWS cloud infrastructure through Infrastructure as Code (IaC).

8. Conclusion

Terraform enables automated and consistent provisioning of cloud infrastructure. In this experiment, an AWS EC2 instance was defined using Terraform configuration files, provisioned using the terraform apply command, and managed through Terraform

commands. The resources created for the experiment were subsequently removed using terraform destroy.

9. Engineering Relevance

The experiment demonstrates Infrastructure as Code practices that improve scalability, reliability, repeatability, and efficiency in cloud-based engineering. Terraform is useful for automating infrastructure deployment and managing cloud resources consistently across development, testing, and production environments.

10. Skills Developed

Students developed skills in writing Terraform configurations, configuring AWS CLI, executing Terraform commands, provisioning AWS resources, retrieving infrastructure outputs, and cleaning up cloud resources.